

Desempenho de Sistemas de Filas Simulados:

Proposta de Experimento Fatorial 2^3 com Replicações
e Análise Exploratória dos Dados

Gruner Antonio Sanchez Morales

Departamento de Computação, Universidade Federal de Ouro Preto

Projeto e Análise de Experimentos Computacionais — Entrega 1

21 de setembro de 2026

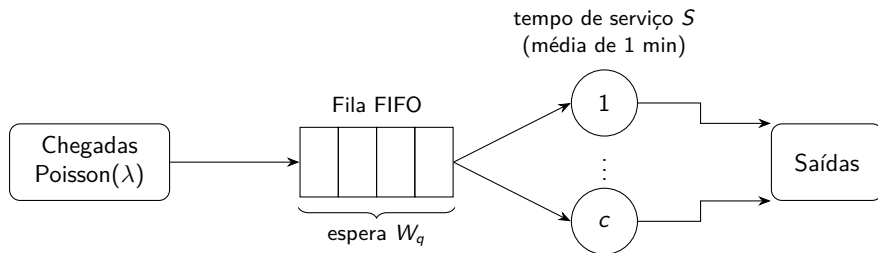
1. Contextualização e problema
2. Objetivo e hipótese geral
3. Estrutura experimental e projeto fatorial 2^3r
4. Fonte dos dados e ambiente computacional
5. Análise exploratória dos dados (piloto)
6. Viabilidade, decisões de projeto e próximos passos

Contextualização: por que estudar filas?

- Filas surgem quando uma demanda variável disputa recursos limitados: requisições em servidores, pacotes em roteadores, processos em uma CPU, clientes em um guichê.
- Decisão típica de projeto: **quantos servidores** alocar para uma carga esperada (Jain, 1991).
- A teoria de filas dá fórmulas fechadas só em casos particulares; a **simulação de eventos discretos** cobre o restante (Law & Kelton, 2000).
- Cada corrida de simulação é aleatória: trata-se de um **experimento computacional**, que exige planejamento (fatores, replicações, aleatorização) (Jain, 1991; Montgomery, 2017).

Figura 1

Sistema de filas modelado



Problema: como o número de servidores, a intensidade de chegadas e a variabilidade do serviço afetam — isolada e conjuntamente — a espera média em fila em regime estacionário?

Relevância: orienta o dimensionamento de capacidade; a espera responde de forma não linear à utilização, o que sugere **interações**.

Objetivo geral. Avaliar, com um projeto fatorial 2^3 e $r = 10$ replicações, o efeito do número de servidores, da taxa de chegada e da distribuição do tempo de serviço sobre a espera média em fila.

Objetivos específicos

1. Estimar efeitos principais e interações, com intervalos de confiança.
2. Determinar a proporção da variação atribuída a cada fator, interação e ao erro.
3. Verificar as suposições do modelo (normalidade, variância constante, independência) e decidir sobre uma transformação.
4. Garantir reprodutibilidade completa: código, sementes e versões registrados.

Definida antes da execução do experimento fatorial:

- Mais servidores (A) $\Rightarrow$ **menor** espera.
- Maior taxa de chegada (B) $\Rightarrow$ **maior** espera.
- Serviço constante, sem variabilidade (C) $\Rightarrow$ **menor** espera.
- Efeitos de A e de C **mais intensos sob alta carga**: interações $A \times B$ e $B \times C$.

Base teórica (fila M/G/1) (Jain, 1991):

$$W_q = \frac{\rho(1 + c_s^2)}{2\mu(1 - \rho)}$$

Expectativa **teórica**, formulada antes das corridas: não vem do piloto e só será avaliada com o fatorial completo (Entrega 2).

Unidade experimental

Uma corrida independente: 50.000 clientes, semente própria, sistema inicialmente vazio.

Variável de resposta

Espera média em fila por corrida (min), após o aquecimento:

$$\overline{W}_q = \frac{1}{N - W} \sum_{i=W+1}^N W_{q,i}$$

com $N = 50.000$ e $W = 1.000$.

Condições constantes

- Tempo médio de serviço: 1 min
- Disciplina FIFO; fila infinita; sem abandono
- Chegadas de Poisson
- Mesmos N , W , código e versões
- Mesma máquina, sem outras cargas pesadas

Tabela 1

Fatores e níveis do experimento

Fator	Descrição	Nível -1	Nível +1
A	Número de servidores (c)	1	2
B	Taxa de chegada (λ , clientes/min)	0,5	0,9
C	Distribuição do tempo de serviço	Exponencial	Constante

Nota. Em ambos os níveis de C a média do serviço é 1 min; a exponencial tem $c_s = 1$ e a constante tem $c_s = 0$.

Em todas as combinações a utilização é $\rho < 1$ (de 0,25 a 0,90): existe regime estacionário.

1. **Sementes independentes:** cada uma das 80 corridas recebe um fluxo próprio com `SeedSequence.spawn` (NumPy), gerador PCG64 (O'Neill, 2014).
2. **Ordem aleatória:** a ordem de execução das 80 corridas é uma permutação uniforme, com semente própria, para não confundir efeitos de ordem com os fatores.
3. **Plano prévio:** o plano completo é gerado e guardado *antes* de qualquer corrida do experimento.

Tabela 2

Combinações experimentais ($r = 10$ por combinação)

Célula	A	B	C	c	λ (cl/min)	Serviço	ρ
1	–	–	–	1	0,5	exponencial	0,50
2	+	–	–	2	0,5	exponencial	0,25
3	–	+	–	1	0,9	exponencial	0,90
4	+	+	–	2	0,9	exponencial	0,45
5	–	–	+	1	0,5	constante	0,50
6	+	–	+	2	0,5	constante	0,25
7	–	+	+	1	0,9	constante	0,90
8	+	+	+	2	0,9	constante	0,45

Nota. $\rho = \lambda / (c\mu)$. Total: $8 \times 10 = 80$ corridas.

Modelo (Jain, 1991)

$$y = q_0 + \sum_j q_j x_j + \sum_{j < k} q_{jk} x_j x_k + q_{ABC} x_A x_B x_C$$

Análise (após as corridas)

- Efeitos e alocação da variação
- ANOVA e IC de 95%
- Diagnóstico de resíduos
- Transformação, se necessário (Box & Cox, 1964)

- Dados **gerados por simulação** de eventos discretos em SimPy (Lünsdorf & Scherfke, s.d.): cada cliente é um processo que solicita um servidor (Resource, fila FIFO).
- Entre chegadas exponenciais; serviço exponencial ou constante; números aleatórios gerados antes da corrida (a semente determina a corrida).
- **Verificação contra a teoria:** Pollaczek–Khinchine (M/M/1, M/D/1) e Erlang C (M/M/2), com 5 corridas de 400.000 clientes em seis combinações. **Erro relativo máximo: 1,3%.**
- Limitação: M/D/2 não tem fórmula fechada simples; usa o mesmo código das combinações verificadas.

Tabela 3

Versões de software

Componente	Versão
Python	3.12.3
SimPy	4.1.2
NumPy	2.4.4
SciPy	1.17.1
pandas	3.0.2
statsmodels	0.15.0
Matplotlib	3.10.8

Nota. Fixadas em `requirements.txt`.

Bibliotecas: SimPy (Lünsdorf & Scherfke, s.d.), NumPy (Harris et al., 2020), SciPy (Virtanen et al., 2020), statsmodels (Seabold & Perktold, 2010), pandas (McKinney, 2010), Matplotlib (Hunter, 2007).

Hardware do piloto: Intel(R) Xeon(R) Processor @ 2.10GHz, 1 núcleo(s), 3,9 GB, Linux 6.18.44-fc-v37.

Ambiente controlado (Sandve et al., 2013; Wilson et al., 2017):

- Versões e sementes registradas
- Execução só por *scripts*, sem passos manuais
- Dados brutos das 80 corridas preservados
- Uma máquina, sem outras cargas
- 80 corridas $\approx$ 36 s; sem APIs nem LLMs

Risco	Como o estudo o evita
Confundir a hipótese com achados da AED	Hipótese formulada <i>antes</i> , a partir da teoria de filas; a AED não a avalia.
Antecipar conclusões do fatorial na AED	Piloto em uma única configuração ; não compara níveis nem estima efeitos.
Réplicas que não são independentes	Cada réplica é uma execução completa (sistema vazio, semente nova); não são lotes de uma corrida nem clientes consecutivos. Independência verificada no piloto.
Execuções incompatíveis com o hardware	$8 \times 10 = 80$ corridas, cerca de 36 s no total; sem APIs nem modelos de linguagem, logo sem cotas.

- Corridas piloto com **sementes diferentes** das do experimento; não entram na análise fatorial.
- **Uma única configuração**, escolhida a priori: célula 3 ($c = 1$, $\lambda = 0,9$, serviço exponencial; $\rho = 0,90$), a mais exigente para dimensionar as corridas (Kelton & Law, 1985; Law & Kelton, 2000).
- 30 corridas independentes, cada uma completa, com $N = 50.000$ e $W = 1.000$.
- Propósito: **caracterizar a resposta e avaliar a viabilidade**.
- Não compara níveis dos fatores, não estima efeitos e **não avalia a hipótese**.

Tabela 4

Médias por corrida no piloto ($n = 30$)

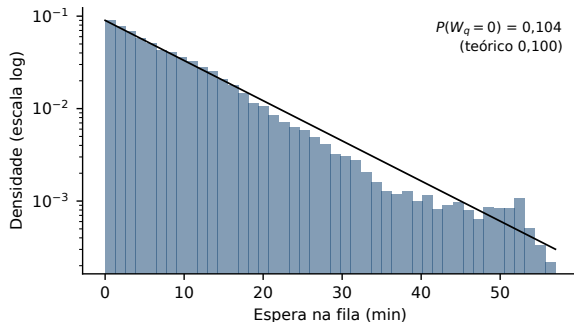
Estatística	Valor
Média (min)	9,112
Desvio-padrão (min)	1,166
Coef. de variação (%)	12,8
Assimetria	1,34
Shapiro–Wilk W (p)	0,891 (0,005)
IC 95% da média (min)	[8,68; 9,55]
Valor teórico (min)	9,00
Erro relativo (%)	1,2

Nota. IC: intervalo de confiança de 95%. Valor teórico: fórmula $M/M/1$. Quartis, mínimo e máximo estão no relatório.

- Média próxima da teoria; o IC contém o valor teórico.
- Dispersão entre corridas grande: CV de 12,8%.
- Distribuição **assimétrica à direita**: algumas corridas com espera média alta.

Figura 2

Esperas individuais (primeira corrida piloto)

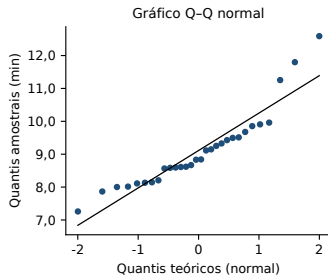
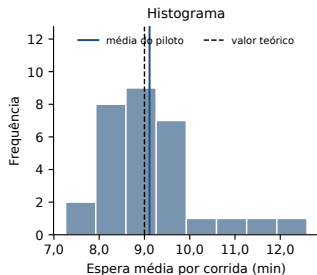


- Massa de probabilidade em zero e cauda exponencial.
- $P(W_q = 0)$ empírica de 10,4% (teórica: 10%).
- Em 1% dos casos a espera passa de 44,2 min.

Nota. Esperas positivas, escala log; linha preta: densidade teórica M/M/1.

Figura 3

Médias por corrida no piloto



- Shapiro–Wilk (Shapiro & Wilk, 1965) **rejeita** a normalidade: $p = 0,005$.
- Q–Q: afastamento na cauda superior.
- Em escala log: assimetria 0,96, $p = 0,058$ (poder limitado).
- Não decide a análise: indica **verificar os resíduos** no experimento.

Figura 4

Média por corrida segundo a ordem das réplicas

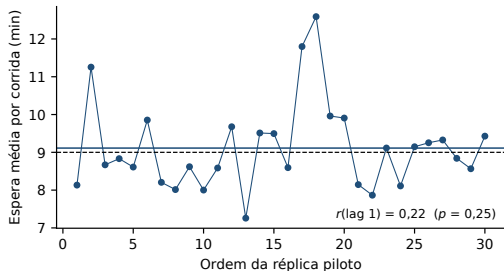
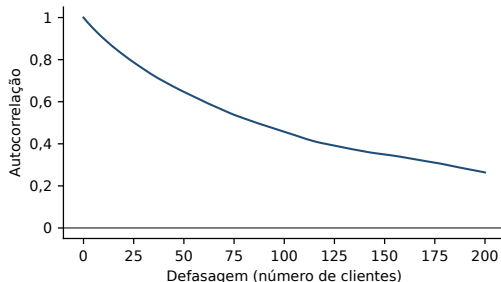


Figura 5

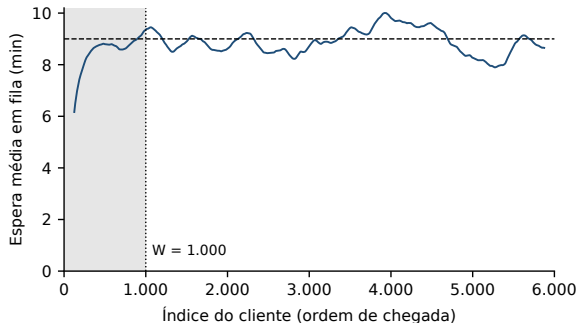
Autocorrelação das esperas individuais



- **Réplicas:** sem tendência; correlação entre réplicas consecutivas $r = 0,22$ ($p = 0,25$). Esperado: cada réplica é uma execução completa com semente própria.
- **Clientes:** autocorrelação de 0,99 (defasagem 1) e 0,90 (defasagem 10): esperas individuais não são independentes; a unidade de análise é a média por corrida.

Figura 6

Procedimento gráfico de Welch



Nota. Média entre 300 réplicas, média móvel de 250 clientes (Welch, 1983). Tracejado: valor teórico; cinza: aquecimento descartado.

- Os 500 primeiros clientes esperam $-16,6\%$ em relação ao platô.
- A curva se estabiliza antes de 1.000 clientes.
- Com $W = 1.000$, a diferença residual é $-0,12 \pm 0,36$ min: indistinguível de zero.

Tabela 5

Comprimento da corrida e dispersão da resposta (piloto, 30 corridas por linha)

N (clientes)	s (min)	CV (%)	Meia-largura do IC ($r = 10$)	Tempo/corrida (s)
10.000	1,508	17,2	$\pm 12,3\%$	0,088
50.000	1,166	12,8	$\pm 9,2\%$	0,445

Nota. Meia-largura do IC de 95% para uma combinação com $r = 10$, em % da média.

Decisões: (1) $N = 50.000$ por corrida; (2) $W = 1.000$ descartados; (3) média por corrida como unidade, com réplicas completas e independentes; (4) verificar normalidade e variâncias dos resíduos no experimento; (5) 80 corridas em cerca de 36 s, compatível com o hardware.

Próximos passos

1. Executar as 80 corridas na ordem sorteada
2. Entrega 2 (26/10): efeitos, interações, IC e testes de significância
3. Diagnóstico dos resíduos
4. Avaliação preliminar da hipótese

Limitações

- Vale para chegadas de Poisson, fila infinita e serviço de média fixa
- $r = 10$ é o mínimo; pode aumentar (corridas baratas)
- M/D/2 sem verificação teórica exata

- Box, G. E. P., & Cox, D. R. (1964). An analysis of transformations. *Journal of the Royal Statistical Society: Series B (Methodological)*, 26(2), 211–252. <https://doi.org/10.1111/j.2517-6161.1964.tb00553.x>
- Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., et al. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95. <https://doi.org/10.1109/MCSE.2007.55>
- Jain, R. (1991). *The art of computer systems performance analysis: Techniques for experimental design, measurement, simulation, and modeling*. John Wiley & Sons.
- Kelton, W. D., & Law, A. M. (1985). The transient behavior of the M/M/c queue, with implications for steady-state simulation. *Operations Research*, 33, 378–396.

- Law, A. M., & Kelton, W. D. (2000). *Simulation modeling and analysis* (3ª ed.). McGraw-Hill.
- Lünsdorf, O., & Scherfke, S. (s.d.). *SimPy: Discrete event simulation for Python* (Versão 4.1.2). <https://simpy.readthedocs.io>
- McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 56–61. <https://doi.org/10.25080/Majora-92bf1922-00a>
- Montgomery, D. C. (2017). *Design and analysis of experiments* (9ª ed.). John Wiley & Sons.
- O'Neill, M. E. (2014). *PCG: A family of simple fast space-efficient statistically good algorithms for random number generation* (Technical report N. HMC-CS-2014-0905). Harvey Mudd College.
- Sandve, G. K., Nekrutenko, A., Taylor, J., & Hovig, E. (2013). Ten simple rules for reproducible computational research. *PLOS Computational Biology*, 9(10), e1003285. <https://doi.org/10.1371/journal.pcbi.1003285>

- Seabold, S., & Perktold, J. (2010). Statsmodels: Econometric and statistical modeling with Python. *Proceedings of the 9th Python in Science Conference*, 92–96.
<https://doi.org/10.25080/Majora-92bf1922-011>
- Shapiro, S. S., & Wilk, M. B. (1965). An analysis of variance test for normality (complete samples). *Biometrika*, 52(3–4), 591–611.
<https://doi.org/10.1093/biomet/52.3-4.591>
- Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., et al. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3), 261–272.
<https://doi.org/10.1038/s41592-019-0686-2>
- Welch, P. D. (1983). The statistical analysis of simulation results. Em S. S. Lavenberg (Ed.), *The computer performance modeling handbook* (pp. 268–328). Academic Press.

Wilson, G., Bryan, J., Cranston, K., Kitzes, J., Nederbragt, L., & Teal, T. K. (2017). Good enough practices in scientific computing. *PLOS Computational Biology*, 13(6), e1005510. <https://doi.org/10.1371/journal.pcbi.1005510>